

LAB-7:IMPERIAL ANALYSIS OF HEAP SORT COMPARE ITS PERFORMANCE WITH MERGE SORT.

THEORY

Heap Sort works by first building a max heap and then repeatedly extracting the maximum element to sort the array. It is an in-place algorithm and does not require extra memory. Merge Sort, on the other hand, uses a divide and conquer approach where the array is recursively divided and then merged in sorted order. It requires additional memory for merging but provides stable and consistent performance.

By running both algorithms on different input sizes (ascending, descending, random), we can observe how their performance grows and compare their efficiency in practice.

PSEUDOCODE

MAX_HEAPIFY(A, n, i):

1. largest = i
2. left = $2*i + 1$
3. right = $2*i + 2$
4. If left < n and $A[\text{left}] > A[\text{largest}]$:
 largest = left
5. If right < n and $A[\text{right}] > A[\text{largest}]$:
 largest = right
6. If largest != i:
 Swap $A[i]$ and $A[\text{largest}]$
 MAX_HEAPIFY(A, n, largest)

HEAP_SORT(A):

1. Build max heap:
 For i from $n//2 - 1$ down to 0:
 MAX_HEAPIFY(A, n, i)
2. For i from n-1 down to 1:
 Swap $A[0]$ and $A[i]$
 Reduce heap size by 1
 MAX_HEAPIFY(A, i, 0)

Time Complexity

Heap Sort

- Best Case: $O(n \log n)$
- Average Case: $O(n \log n)$
- Worst Case: $O(n \log n)$

Merge Sort

- Best Case: $O(n \log n)$
- Average Case: $O(n \log n)$
- Worst Case: $O(n \log n)$

OUTPUT

```
023bscit009_binesh
For input n:: 1000
Ascending (Heap Sort):: Count:: 69122
Random (Heap Sort):: Count:: 64087
Descending (Heap Sort):: Count:: 60544
```

```
023bscit009_binesh
For input n:: 5000
Ascending (Heap Sort):: Count:: 431785
Random (Heap Sort):: Count:: 402827
Descending (Heap Sort):: Count:: 385406
```

```
023bscit009_binesh
For input n:: 10000
Ascending (Heap Sort):: Count:: 934239
Random (Heap Sort):: Count:: 875929
Descending (Heap Sort):: Count:: 840161
```

Data Table:

Input Size	Ascending	Random	Descending
1,000	69,122	64,087	60,544
2,000	151,658	142,607	135,503
3,000	243,134	226,492	215,286
4,000	334,285	312,641	299,330
5,000	431,785	402,827	385,406
6,000	530,015	495,744	472,909
7,000	628,470	588,389	563,257
8,000	726,566	681,625	653,936
9,000	829,829	777,957	746,629
10,000	934,239	875,929	840,161

Function Growth:

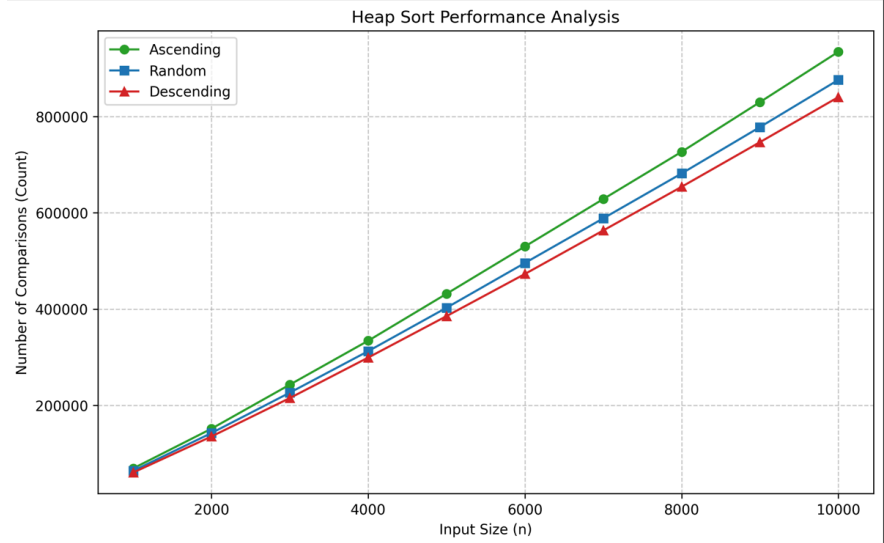
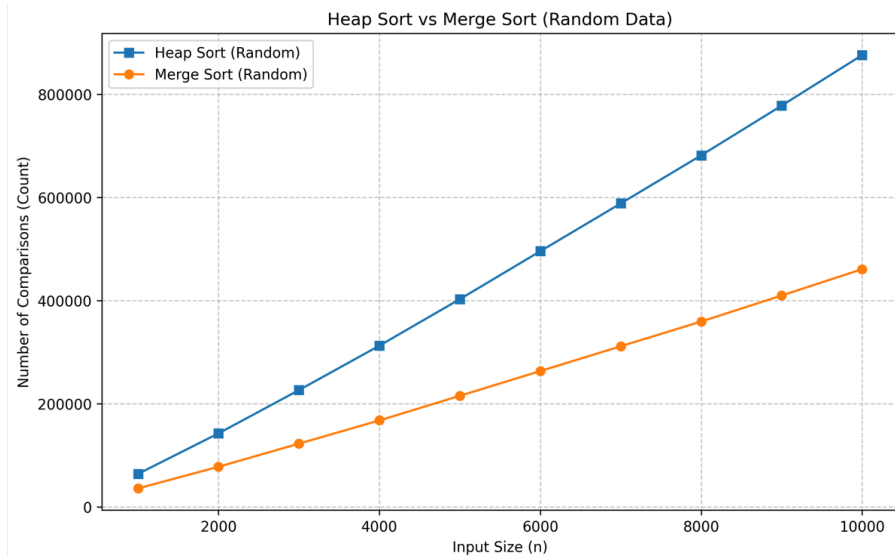
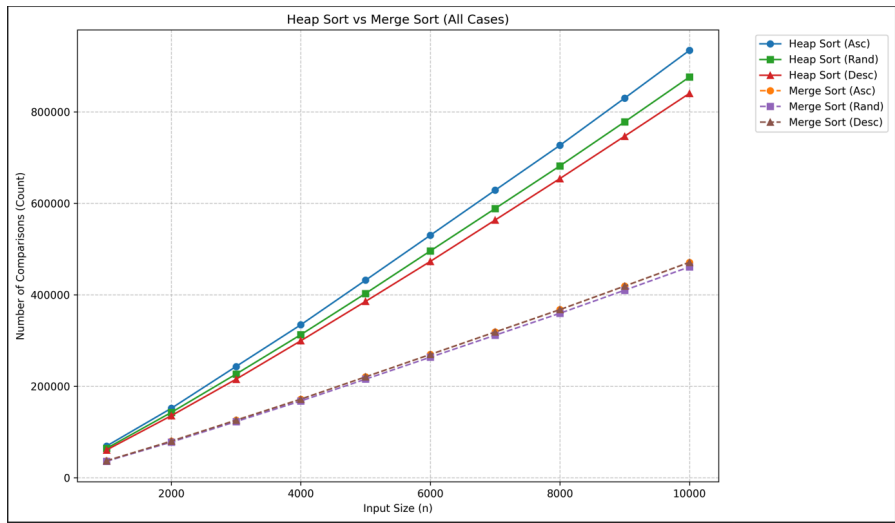


Fig: Heapsort Performance Analysis

Heap v.s Merge:



CONCLUSION

Both Heap Sort and Merge Sort have the same theoretical time complexity of $O(n \log n)$, but their practical performance differs. Merge Sort generally performs better due to its sequential memory access and stability, while Heap Sort may have slightly higher constant factors due to heap operations. However, Heap Sort is more space-efficient as it is an in-place algorithm, whereas Merge Sort requires additional memory. The empirical analysis shows that although both algorithms scale similarly, Merge Sort is often preferred in practice for its consistency and efficiency.